

Everywhere Cursor: Extending Desktop Mouse Interaction into Spatial Augmented Reality

Daekun Kim
d332kim@uwaterloo.ca
University of Waterloo
Canada

Daniel Vogel
dvogel@uwaterloo.ca
University of Waterloo
Canada

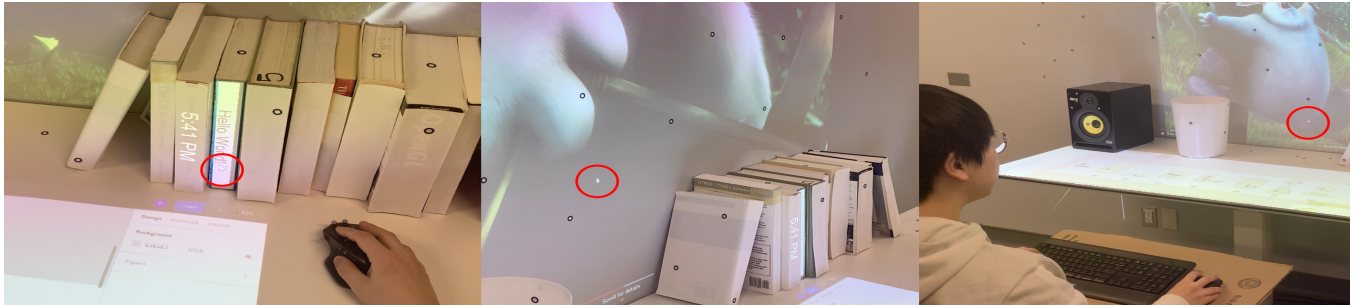


Figure 1: *Everywhere Cursor* extends desktop cursor behaviour into SAR while being adaptable to different surface geometries: (a) mouse interaction on an irregular surface, (b) desktop-like cursor behaviour on planar surfaces, (c) a user interacting with contents throughout the room using the mouse.

ABSTRACT

Spatial augmented reality (SAR) allows us to extend desktop environments to our surroundings. However, previous works on enabling mouse interaction in SAR lacks consistent pointing behaviour with desktop environments. We present *Everywhere Cursor*, an indirect mouse interaction technique that moves along the environment’s surface geometry, which preserves mouse behaviour in desktop environments. We discuss fundamental design problems in extending desktop mouse interaction to SAR, implementation details of our system, and guidelines on future improvements. Our work provides an important step in the transition of desktop environments to a pervasive computing system.

CCS CONCEPTS

• **Human-centered computing** → **Interaction techniques.**

KEYWORDS

interaction techniques

ACM Reference Format:

Daekun Kim and Daniel Vogel. 2022. Everywhere Cursor: Extending Desktop Mouse Interaction into Spatial Augmented Reality. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts (CHI '22 Extended Abstracts)*, April 29-May 5, 2022, New Orleans, LA, USA. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3491101.3519796>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CHI '22 Extended Abstracts, April 29-May 5, 2022, New Orleans, LA, USA

© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM. ACM ISBN 978-1-4503-9156-6/22/04...\$15.00

<https://doi.org/10.1145/3491101.3519796>

1 INTRODUCTION

Spatial Augmented Reality (SAR) is a method to render digital content directly onto surfaces in a real physical 3D environment, typically using projection mapping [20, 21]. It can cover surfaces and objects with illusionary textures and virtual 3D objects, enabling applications like gaming and teleportation (e.g., [10, 16]). But SAR can also be used in a more subtle and integrated way, to selectively augment real surfaces and objects with 2D “surface mapped” digital information [7]. Essentially, every surface becomes a potential computer display without the need for special AR glasses.

A compelling use case for surface mapped SAR is to extend the display space for traditional desktop computing, making it pervasive and integrated into the surrounding environment. Potential use cases include: increased screen space [11]; spatial anchoring of information [17]; interactive ambient displays [8]; and sharing of desktop information between digital devices [22, 23]. Imagine using a nearby plant pot as a cylindrical ambient notification display, or storing favorite file folders on the spines of books on a shelf.

A key decision when extending desktop computing into a surface mapped SAR environment is what pointing method to use. Direct touch would only be reasonable for nearby targets and it can be less precise [24]. Mid-air raycasting using hand tracking or 3D controllers have issues like tremor, low input precision on distant targets, and 3D surface occlusion [2, 25]. But the mouse is accurate, fast, and comfortable [5, 26] and since it is a relative indirect method, in theory there is no limitation in terms of target distance or surface visibility. Moreover, unlike touch and raycasting, the mouse is already familiar in desktop computing so also using it when interacting in the surrounding SAR environment would make desktop and SAR feel integrated.

Previous works have suggested mouse-to-cursor mapping for limited types of SAR. Although some enable indirect mouse pointing [12, 23], they only support planar interactions with tabletops

and walls. More recent works investigate a generalized approach for mouse interaction with arbitrary surfaces in SAR [6, 13]. However, they perform cursor placement using raycasting, albeit not from the user's hand (e.g., user's head, ceiling mirror). Therefore, they share disadvantages with raycasting, such as low input precision on distant targets and sensitivity to surface occlusion, and changing the fundamental mouse pointing paradigm limits interaction consistency of the mouse in SAR with respect to traditional desktop.

We contribute *Everywhere Cursor*¹, a mouse interaction technique for SAR that closely resembles pointing behaviour in conventional desktop computing. We identify four design challenges: maintaining upright orientation, obtaining intuitive and smooth cursor trajectory, handling crevices and cliff-like regions to balance control and speed, and interaction with occluded surfaces. Our proof-of-concept prototype addresses the first three of these fundamental issues with a novel geometry-based cursor movement algorithm, with occluded interactions left for future work. We provide a detailed description of our technical approach with illustrative application demonstrations, and we discuss further improvements, ideas for resolving the remaining challenge, and an outline for a future user study.

2 RELATED WORK

Our approach is inspired by works that extend personal computing to non-desktop display environments. Rekimoto and Saitoh [23] integrate digital devices (e.g., laptops and PDAs) and projection surfaces together, enabling common mouse interactions like drag-and-drop. While they demonstrate a smooth cursor movement between laptop monitor and projection areas, their system only operates with planar projections, such as tablespots and walls, which limits its generalizability. Nakashima et al. [15] introduce a tabletop multi-user mouse technique that aligns the cursor's X- and Y-axes with the user's orientation on a single tabletop surface. Other systems [11, 12, 18] extend desktops into multiple displays and projection areas but do not discuss the specifics of cross-display mouse interaction or mainly uses direct touch as the input method.

Extending a mouse into to SAR necessitates the extension of mouse interaction to arbitrary geometric surfaces. Marner et al. [13] map mouse movement to a point in the user's view plane by raycasting from the head position to find the cursor position on a 3D surface. Gervais et al. [6] present a similar mouse technique for SAR, but restricted to objects placed in front of a monitor. They share disadvantages with raycasting, such as low input precision on distant targets, as well as sensitivity to surface occlusion. More critically, a raycasting-based cursor mapping solely relies on the user's perspective. For instance, given two surfaces facing different directions or placed at varying depths from the user, the cursor can change in linear speed even when constant mouse velocity is maintained, and it may even skip parts of the surfaces that are occluded from the user's point of view. In desktop, the cursor moves along the plane of the monitor, which can be thought of as surface geometry in the environment. Moreover, the control-display gain does not vary with the user's position. A discrepancy in cursor movement behaviour limits interaction consistency with a traditional desktop.

¹The name is homage to Pinhanez's Everywhere Display [18].

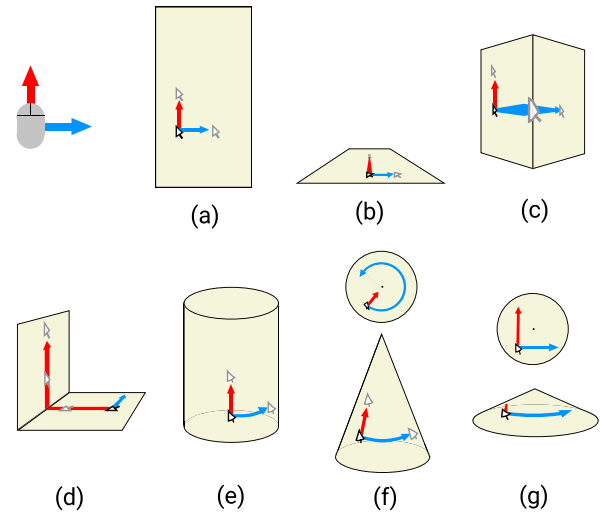


Figure 2: Possible surface geometries in SAR and ideal cursor travel path: (a) wall, (b) tabletop, (c) convex corner, (d) concave corner, (e) cylinder, (f) steep cone, (g) flat cone.

While distinctly unique from SAR, multi-display environments can also provide similar experience by surrounding the user with digital content. Johanson et al. [9] and Pirschheim et al. [19] enable cursor movement across separated planar displays that are not necessarily co-planar using stitching and warping. Nacenta et al. [14] move the cursor along a virtual sphere centered around the user's head using raycasting. This makes cursor movement fully independent from surface geometry which introduces many of the same consistency and precision issues with raycast techniques. Xiao et al. [27] describe a related mouse technique, but raycast through a ceiling-centered hemisphere point to find the cursor position, and visualization of cursor even on non-monitor areas using projection. While these works help inform and motivate our work, they only accommodate planar rectangular screens in a limited range of orientations.

Our work achieves mouse pointing experience for SAR that is consistent with a traditional desktop using geometry-based cursor movement that operates on any arbitrary surface configuration, including curved or jagged surfaces.

3 DESIGN CONSIDERATIONS

We propose four challenges. The overall goal is to recreate cursor behaviour from desktop in a way that preserves its speed, accuracy, and control, while enabling universal access to all surfaces in SAR environment when designing mouse-to-cursor mapping for SAR.

3.1 Intuitive Orientation and Movement Direction

On a desktop computer, the cursor points in a consistent "upright" orientation as it moves in a predictable up-down and left-right motion along the plane of the monitor. Our goal is to extend the notion of an intuitive upright orientation and movement to any

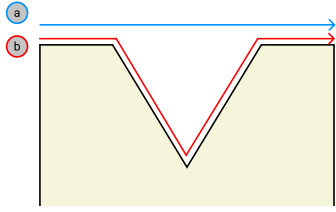


Figure 3: Two potential ways of travelling through a crevice: (a) skipping across the canyon (b) moving along the concave surface.

arbitrary surface. On a flat surface, extending such behaviour to SAR is trivial since it is exactly like in desktop. However, obtaining explicit definition of generalized upright orientation for any arbitrary surfaces is difficult. To achieve this, we can examine a few examples and infer from our findings. From Figure 2, we see that generally speaking, cursor should be pointing along the global up-axis on vertical surfaces and the user’s forward-axis on horizontal surfaces. For the latter case, the forward-axis should not be a global one but one that is based off of the viewpoint of the user, because it changes drastically depending on where the user is facing.

3.2 Smooth Cursor Trajectory

While the path should ideally follow the surface geometry, small bumps and scan errors should not introduce jagged movement behaviour. Moreover, given a constant mouse velocity, cursor velocity must stay as linear as possible on the path. For instance, a corner should not introduce sudden change in cursor velocity.

3.3 Handling Crevices and Cliffs to Balance Control and Speed

Solutions to the previous problems enable access to any part of the environment including its fine details, but there are cases when speed is preferred over detailed interaction. A SAR environment often contains empty regions (“gaps”), deep deviations in geometry (“crevices”), and sudden changes in surface depth (“cliffs”) due to arrangement of objects or scan inaccuracy. There needs to be a way to selectively skip across these features without forcing a long movement path. However, the user also needs to be able to access regions between crevices or side of a cliff. Consider a V-shaped crevice (Figure 3). A user may want to access contents in the concave area, or they may want to skip across it. We want to allow access to all parts of the environment including those in such a crevice, while ensuring fast cursor mobility in the general case. For example, imagine two books in a bookshelf separated with a crevice. A user could be only interested in interacting with the spines for file folder placement, or may want to use the book cover as a means to peak what is stored in the folders.

3.4 Interaction with Occluded Surfaces

Naturally, due to arrangement of objects and surface geometries, there will be surfaces in the environment that the user cannot see. Interactions with partially- or fully-occluded surfaces is crucial for universal access to contents throughout the environment. There are two considerations for achieving this. Firstly, travelling to occluded

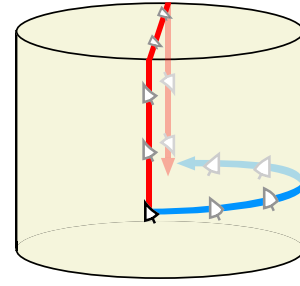


Figure 4: Ambiguous cursor orientation: Natural transition of cursor orientation in two paths appear to contradict.

surfaces is challenging to generalize. We must ensure that the cursor orientation remains sensible and consistent with respect to the first problem above, particularly during transition of the cursor between visible and occluded regions (Figure 4). Secondly, despite occlusion, the user should be able to view the occluded content and cursor. While the user can maneuver themselves around the environment to bring hidden content into in their line of sight, it is possible that the occluded surface is not even part of the projection area, due to limitations in the arrangement of projectors. Even in that extreme case, occluded surfaces should be accessible as spatially associated information storage, or as intuitive locations for configuration interfaces (*e.g.*, adjust a weather display on a coffee cup by moving the cursor to the back surface of the cup).

4 EVERYWHERE CURSOR TECHNIQUE

We briefly describe *Everywhere Cursor*, our prototype that addresses the challenges of upright cursor orientation, intuitive and smooth movement trajectory, and movement across crevices and cliffs. Our room-scale projection mapping is similar to other SAR systems like RoomAlive [10]. Three 1920×1200 60-FPS projectors along with three cameras, all connected to a PC with Quadro RTX 5000 are used to display digital content with a layout that minimizes occlusion while maintaining 10% overlap between the projections, as recommended for RoomAlive [10]. The content of our software is generated using C# and Unity, where the position, rotation, and view frustum of each virtual camera in the game scene corresponds to a projector pose. The projectors are calibrated using Christie Mystique software based on a static 3D scan of the room with 2mm resolution. *Smart UV project* feature [1] of Blender computer graphics software is used to generate the texture UV mapping of the scan. We intend to incorporate dynamic room scanning in the future using depth cameras such as Microsoft Kinect. A ten-camera Vicon motion tracking system tracks the user’s head as well as the mouse using 6.4mm markers.

4.1 Cursor Projector

For cursor visualization, we use a small virtual orthogonal projector, called the *cursor projector*, that floats throughout the 3D model of the room. It renders a cursor 1cm in width and 1.5cm in height when cast perpendicular to the surface (Figure 5a). From its top left corner, a raycast is performed to send UI events like clicking,

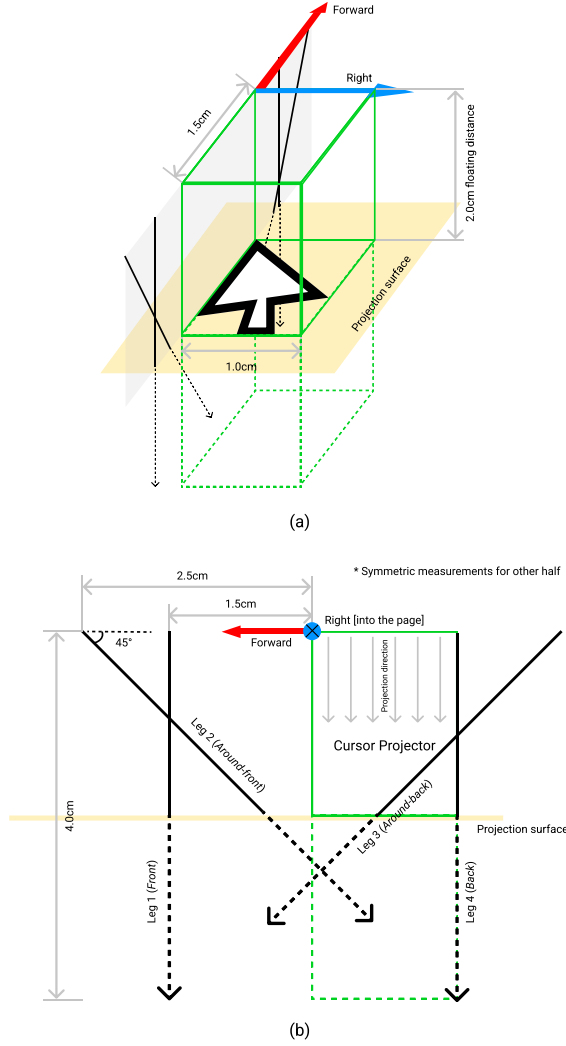


Figure 5: Structure of the cursor projector and its legs along the plane of movement, viewed (a) in 3D (b) from the side. Dotted lines on the legs are under the surface.

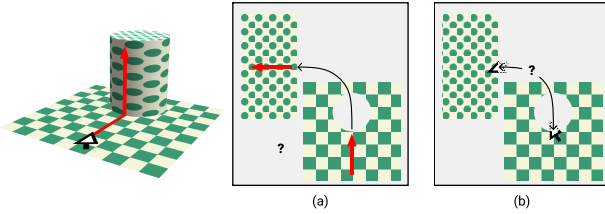


Figure 6: Problems with placing cursor on texture map: (a) Movement across surfaces that are isolated in the texture space (b) Splitting cursor image across two regions.

dragging, and scrolling to the Unity UI system, so any Unity UI component is compatible with our technique.

We justify the decision to use a cursor projector as follows. In SAR, any content that is not directly mapped onto the environment geometry results in a blurry image [10]. An intuitive method of 2D content placement is texture mapping. A naive approach would move the cursor on the 2D texture map of the SAR environment. However, this presents two issues. Firstly, when the cursor is moving across two surfaces that share an edge, the automatically-generated texture map regions may be isolated (Figure 6a). In general there is no straightforward way of finding a path in texture space to move the cursor across two isolated texture patches. Even tediously mapping shared transition edges for every pair of coinciding regions does not encode required trajectory changes related to challenge 2. Secondly, when the cursor visualization straddles two or more surfaces, such as around corners, resolving which portion of the cursor image should go to which texture map region is difficult in texture space (Figure 6b). Instead, using a virtual cursor projector allows cursor placement to be based on the 3D model of the environment itself rather than on its texture map, which removes these complexities.

4.2 Staying Afloat Parallel to Surface

We employ a series of 4cm-long raycasts, called *legs*, around the cursor that detect nearby geometry (Figure 5b). The legs lend themselves intuitively to a *bug* analogy for cursor locomotion; we can think of the cursor projector as a small bug crawling throughout the SAR environment while naturally orienting and distancing itself away from the surface. These legs are not visible to the user, and they are always placed along the direction of movement. They serve two purposes: orienting the cursor projector perpendicular to surrounding geometry, and maintaining a floating distance away from the surface. We found a floating distance of 2cm yields a good balance between consistent projection and smooth transition of cursor orientation. The legs give rise to a *geometry-based* cursor movement. Because surrounding polygons have varying normals, we take their weighted average $\vec{n}_{avg} = \sum \frac{\hat{n}_i}{\|\vec{p}_i - \vec{c}\|}$, where $\vec{p}_i$, $\hat{n}_i$ are collision point and normal at the i th leg's raycast endpoint, respectively, and $\vec{c}$ is the center of the cursor projector. Therefore, closer normals have higher weight. The cursor projector's up vector is set to $\vec{n}_{avg}$. Lastly, the cursor projector moves along $\vec{n}_{avg}$ to maintain the floating distance away from the surface below.

4.3 Upright Cursor Orientation

We resolve the “upright” orientation challenge by first determining whether the nearby surfaces are horizontal or vertical, and setting the appropriate forward direction. We define the cursor surface as *horizontal* if the angle between $\vec{n}_{avg}$ and global up-vector is less than 45°, and *vertical* if otherwise. In the horizontal case, the forward vector of the mouse (as an approximation of the user's forward vector) is projected onto a plane with normal $\vec{n}_{avg}$, and the resulting vector from the projection is set to be the forward direction of the cursor projector. The vertical case is identical, except the global up vector is used in place of the mouse's forward vector. This way, the cursor behaves exactly like in a desktop computer when sitting on a wall or on a tabletop, but it can still move smoothly across arbitrary surfaces since the orientation is well-defined for any type of geometry.

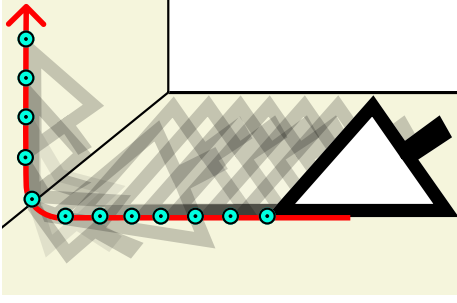


Figure 7: Cursor trajectory upon upward mouse motion. Each cyan dot represents evaluation points for next cursor position and orientation, which results in a path that is sensible and smooth as desired.

4.4 Formation of Cursor Trajectory

The interaction technique naturally forms a smooth and intuitive cursor trajectory without performing a complicated path pre-computation by narrowing the problem to locally-determining next cursor pose at each fixed-length step (Figure 7). The mouse delta along its X- and Y-axes are mapped to cursor projector movement along its local right and forward vectors, respectively. The cursor movement is subdivided into points 1mm apart, where we perform the cursor reorientation and floating distance operation. The 16 Filter is applied to the cursor’s position and orientation at every frame to prevent jitter and hysteresis [4]. While we did not perform systematic tuning of the filter parameters, we have found $f_{min} = 1, \beta = 1$ yield minimal jitter and lag through trial and error.

4.5 Jumping Across Crevices and Cliffs

We allow bypassing of obstructing regions like crevices and cliffs by adopting a speed-based mixing of two movement paths: one that is described above (*geometry-based*), and the other that is determined independent of surface geometry (*inertia-based*). The inertia-based path is along the cursor right and forward vectors, and the cursor projector does not change orientation. Rather, it behaves as if the cursor projector plane of movement is fixed. A vertical raycast in the down and up cursor plane surface normal (in that order) is performed, and the first hit is used as the surface for the cursor projector to float on, making the cursor appear to *stick to* a surface along its vertical axis.

The resulting positions from the geometry-based path and the inertia-based path are linearly interpolated, and resulting orientations are spherically interpolated. The interpolation parameter is obtained from $t(\vec{v}) = \frac{1}{1+e^{-k(\|\vec{v}\|-c)}} \in (0, 1)$, which is a logistic function in terms of mouse delta vector $\vec{v} \in \mathbb{R}^2$. Parameters k and c define growth rate and horizontal center of the function curve, respectively. While we did not perform a formal investigation of optimal parameter values, we found $k = 2, c = 4$ allow easy access to each type of paths while minimizing accidental activation.

5 DISCUSSION

Everywhere Cursor demonstrates a consistent mouse pointing behaviour between SAR and desktop. We discuss potential use cases

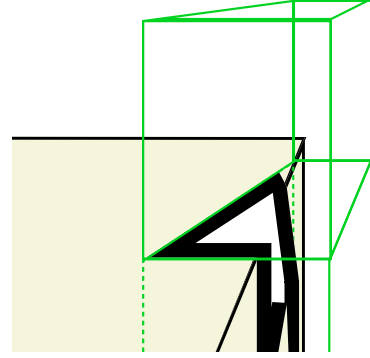


Figure 8: Cursor projector around a corner.

of our technique, resolving the remaining challenges, and an outline for a future user study.

5.1 Applications

We are interested in applications that our technique enables for desktop-anchored pervasive computing. For instance, designing content for large projection mapping installations is currently performed by manipulating projector images (e.g., using Adobe After Effects) or environment textures (e.g., using Blender), both in a 2D desktop monitor. With Everywhere Cursor, a designer could move fluidly between monitor and environment, and manipulate content directly in the projection mapped scene. They can also outline the contents on desktop monitor first, and make detailed live onsite touch-ups later in the full size projection.

Another application example is an environment filled with web pages or desktop applications, similar to a multi-display environment, using standard-feeling mouse interactivity without any UI redesign.

5.2 Limitations

While Everywhere Cursor presents an exciting opportunity for expanding typical desktop interaction to SAR, it has limitations in its visualization that deviate from cursor behaviour in desktop. Notably, the cursor image becomes distorted around corners due to the nature of projections (Figure 8). Ideally, the cursor image would have same area regardless of surface geometry. A potential solution is to divide the cursor projector into a grid such that each mini-projector stays perpendicular to its nearest projection surface. For instance, in the corner example from Figure 8, one set of mini-projectors would point perpendicular to the horizontal surface, and the other set to the vertical surface.

We also did not address interactions with occluded surfaces. Awareness of cursor position as well as visualization of contents on occluded geometry could allow mouse interaction with little disadvantage compared to pointing on visible surfaces. With this in mind, making occluding surfaces semi-transparent or displaying a surrounding halo [3] on visible surfaces around the cursor could be good solutions. Additional research should explore this type of interaction in SAR.

Lastly, our technique does not handle movement across gaps in the environment. While restricting the cursor from accessing such regions may appear to be a good solution initially, a pilot testing showed that distinguishing gaps from crevices and cliffs is difficult to achieve reliably, and the user can perceive such behaviour to be jarring. Instead, we propose displaying a halo around the cursor as described above whenever the cursor moves out of the visible region.

5.3 Future Work

While we did not perform an experimental evaluation, future work could explore how our technique compares to other existing mouse-to-SAR cursor mapping techniques, such as the raycasting-based techniques like Viewpoint Cursor [13] and Tangible Viewports [6]. Even comparisons with other perspective-based techniques for multi-display environments like Perspective Cursor [14] and Ubiquitous Cursor [27] could be interesting if adapted appropriately to SAR. We hypothesize that our technique will yield more intuitive experience with traditional desktop and web applications placed in SAR; our technique closely resembles cursor movement in a traditional desktop, whereas others behave distinctly from desktop's monitor-centric behaviour, so may not be suitable for UI with desktop interaction in mind.

6 CONCLUSION

Everywhere Cursor facilitates geometry-based mouse pointing behaviour in SAR that is consistent with traditional desktops. Cursor movement follows a sensible and continuous path while maintaining orientation perpendicular to its nearby geometry as expected from cursor behaviour on a planar monitor. Interaction with both fine details and fast movement across obstructing regions like crevices and cliffs are both possible. Future work can address interaction with occluded surfaces, evaluation of our technique against existing cursor mapping technique for SAR, and examine new applications that fluid integration of desktop interaction in SAR can enable. With our work, we envision a future where traditional desktop computing extends beyond our monitors and into the world around us.

ACKNOWLEDGMENTS

This work made possible by NSERC Discovery Grant 2018-05187, Canada Foundation for Innovation Infrastructure Fund 33151 "Facility for Fully Interactive Physio-digital Spaces," Ontario Early Researcher Award ER16-12-184, NSERC USRA Program for undergraduate research, and the Cheriton School of Computer Science Undergraduate Research Fellowship.

REFERENCES

- [1] [n.d.]. Mapping Types. https://docs.blender.org/manual/en/2.79/editors/uv_image/uv/editing/unwrapping/mapping_types.html
- [2] Marc Baloup, Thomas Pietrzak, and G ry Casiez. 2019. *RayCursor: A 3D Pointing Facilitation Technique Based on Raycasting*. Association for Computing Machinery, New York, NY, USA, 1–12. <https://doi.org/10.1145/3290605.3300331>
- [3] Patrick Baudisch and Ruth Rosenholtz. 2003. Halo: A Technique for Visualizing off-Screen Objects. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Ft. Lauderdale, Florida, USA) (CHI '03). Association for Computing Machinery, New York, NY, USA, 481–488. <https://doi.org/10.1145/642611.642695>
- [4] G ry Casiez, Nicolas Roussel, and Daniel Vogel. 2012. 1   Filter: A Simple Speed-Based Low-Pass Filter for Noisy Input in Interactive Systems. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Austin, Texas, USA) (CHI '12). Association for Computing Machinery, New York, NY, USA, 2527–2530. <https://doi.org/10.1145/2207676.2208639>
- [5] Renaud Gervais, J r my Frey, and Martin Hachet. 2015. Pointing in Spatial Augmented Reality from 2D Pointing Devices. In *Human-Computer Interaction – INTERACT 2015*, Julio Abascal, Simone Barbosa, Mirko Fetter, Tom Gross, Philippe Palanque, and Marco Winckler (Eds.). Springer International Publishing, Cham, 381–389.
- [6] Renaud Gervais, Joan Sol Roo, and Martin Hachet. 2016. Tangible viewports: Getting out of flatland in desktop environments. *TEI 2016 - Proceedings of the 10th Anniversary Conference on Tangible Embedded and Embodied Interaction* (2016), 176–184. <https://doi.org/10.1145/2839462.2839468>
- [7] Jeremy Hartmann and Daniel Vogel. 2021. An examination of mobile phone pointing in surface mapped spatial augmented reality. *International Journal of Human-Computer Studies* 153 (2021), 102662. <https://doi.org/10.1016/j.ijhcs.2021.102662>
- [8] Hiroshi Ishii, Craig Wisneski, Scott Brave, Andrew Dahley, Matt Gorbet, Brygg Ullmer, and Paul Yarin. 1998. AmbientROOM: Integrating Ambient Media with Architectural Space. In *CHI 98 Conference Summary on Human Factors in Computing Systems* (Los Angeles, California, USA) (CHI '98). Association for Computing Machinery, New York, NY, USA, 173–174. <https://doi.org/10.1145/286498.286652>
- [9] Brad Johanson, Greg Hutchins, Terry Winograd, and Maureen Stone. 2002. PointRight: Experience with Flexible Input Redirection in Interactive Workspaces. In *Proceedings of the 15th Annual ACM Symposium on User Interface Software and Technology* (Paris, France) (UIST '02). Association for Computing Machinery, New York, NY, USA, 227–234. <https://doi.org/10.1145/571985.572019>
- [10] Brett Jones, Rajinder Sodhi, Michael Murdock, Ravish Mehra, Hrvoje Benko, Andrew Wilson, Eyal Ofek, Blair MacIntyre, Nikunj Raghuvanshi, and Lior Shapira. 2014. RoomAlive: Magical Experiences Enabled by Scalable, Adaptive Projector-Camera Units. In *Proceedings of the 27th Annual ACM Symposium on User Interface Software and Technology* (Honolulu, Hawaii, USA) (UIST '14). Association for Computing Machinery, New York, NY, USA, 637–644. <https://doi.org/10.1145/2642918.2647383>
- [11] Shaun K. Kane, Daniel Avrahami, Jacob O. Wobbrock, Beverly Harrison, Adam D. Rea, Matthai Philipose, and Anthony LaMarca. 2009. Bonfire: A Nomadic System for Hybrid Laptop-Tabletop Interaction. In *Proceedings of the 22nd Annual ACM Symposium on User Interface Software and Technology* (Victoria, BC, Canada) (UIST '09). Association for Computing Machinery, New York, NY, USA, 129–138. <https://doi.org/10.1145/1622176.1622202>
- [12] Blair MacIntyre, Elizabeth D. Mynatt, Stephen Volda, Klaus M. Hansen, Joe Tullio, and Gregory M. Corso. 2001. Support for Multitasking and Background Awareness Using Interactive Peripheral Displays. In *Proceedings of the 14th Annual ACM Symposium on User Interface Software and Technology* (Orlando, Florida) (UIST '01). Association for Computing Machinery, New York, NY, USA, 41–50. <https://doi.org/10.1145/502348.502355>
- [13] Michael R. Marner, Ross T. Smith, and Bruce H. Thomas. 2015. Mapping 2D input to 3D immersive spatial augmented reality. In *2015 IEEE Symposium on 3D User Interfaces (3DUI)*. 171–172. <https://doi.org/10.1109/3DUI.2015.7131755>
- [14] Miguel A. Nacenta, Samer Sallam, Bernard Champoux, Sriram Subramanian, and Carl Gutwin. 2006. Perspective Cursor: Perspective-Based Interaction for Multi-Display Environments. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Montr al, Qu bec, Canada) (CHI '06). Association for Computing Machinery, New York, NY, USA, 289–298. <https://doi.org/10.1145/1124772.1124817>
- [15] Kousuke Nakashima, Takashi Machida, Kiyoshi Kiyokawa, and Haruo Takemura. 2005. A 2D-3D Integrated Environment for Cooperative Work. In *Proceedings of the ACM Symposium on Virtual Reality Software and Technology* (Monterey, CA, USA) (VRST '05). Association for Computing Machinery, New York, NY, USA, 16–22. <https://doi.org/10.1145/1101616.1101621>
- [16] Tomislav Pej sa, Julian Kantor, Hrvoje Benko, Eyal Ofek, and Andrew Wilson. 2016. Room2Room: Enabling Life-Size Telepresence in a Projected Augmented Reality Environment. In *Proceedings of the 19th ACM Conference on Computer-Supported Cooperative Work & Social Computing* (San Francisco, California, USA) (CSCW '16). Association for Computing Machinery, New York, NY, USA, 1716–1725. <https://doi.org/10.1145/2818048.2819965>
- [17] Simon T. Perrault, Eric Lecolinet, Yoann Pascal Bourse, Shengdong Zhao, and Yves Guard. 2015. Physical Loci: Leveraging Spatial, Object and Semantic Memory for Command Selection. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems* (Seoul, Republic of Korea) (CHI '15). Association for Computing Machinery, New York, NY, USA, 299–308. <https://doi.org/10.1145/2702123.2702126>
- [18] Claudio S. Pinhanez. 2001. The Everywhere Displays Projector: A Device to Create Ubiquitous Graphical Interfaces. In *Proceedings of the 3rd International Conference on Ubiquitous Computing* (Atlanta, Georgia, USA) (UbiComp '01). Springer-Verlag, Berlin, Heidelberg, 315–331.

- [19] Christian Pirchheim, Manuela Waldner, and Dieter Schmalstieg. 2009. Deskothèque: Improved Spatial Awareness in Multi-Display Environments. In *2009 IEEE Virtual Reality Conference*. 123–126. <https://doi.org/10.1109/VR.2009.4811010>
- [20] Ramesh Raskar, Jeroen van Baar, Paul Beardsley, Thomas Willwacher, Srinivas Rao, and Clifton Forlines. 2003. ILamps: Geometrically Aware and Self-Configuring Projectors. *ACM Trans. Graph.* 22, 3 (jul 2003), 809–818. <https://doi.org/10.1145/882262.882349>
- [21] Ramesh Raskar, Greg Welch, Matt Cutts, Adam Lake, Lev Stesin, and Henry Fuchs. 1998. The Office of the Future: A Unified Approach to Image-Based Modeling and Spatially Immersive Displays. In *Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '98)*. Association for Computing Machinery, New York, NY, USA, 179–188. <https://doi.org/10.1145/280814.280861>
- [22] Jun Rekimoto. 1997. Pick-and-Drop: A Direct Manipulation Technique for Multiple Computer Environments. In *Proceedings of the 10th Annual ACM Symposium on User Interface Software and Technology (Banff, Alberta, Canada) (UIST '97)*. Association for Computing Machinery, New York, NY, USA, 31–39. <https://doi.org/10.1145/263407.263505>
- [23] Jun Rekimoto and Masanori Saitoh. 1999. Augmented Surfaces: A Spatially Continuous Work Space for Hybrid Computing Environments. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (Pittsburgh, Pennsylvania, USA) (CHI '99)*. Association for Computing Machinery, New York, NY, USA, 378–385. <https://doi.org/10.1145/302979.303113>
- [24] Katie A. Siek, Yvonne Rogers, and Kay H. Connelly. 2005. Fat Finger Worries: How Older and Younger Users Physically Interact with PDAs. In *Proceedings of the 2005 IFIP TC13 International Conference on Human-Computer Interaction (Rome, Italy) (INTERACT'05)*. Springer-Verlag, Berlin, Heidelberg, 267–280. https://doi.org/10.1007/11555261_24
- [25] Daniel Vogel and Ravin Balakrishnan. 2005. Distant Freehand Pointing and Clicking on Very Large, High Resolution Displays. In *Proceedings of the 18th Annual ACM Symposium on User Interface Software and Technology (Seattle, WA, USA) (UIST '05)*. Association for Computing Machinery, New York, NY, USA, 33–42. <https://doi.org/10.1145/1095034.1095041>
- [26] Stephen Voda, Mark Podlaseck, Rick Kjeldsen, and Claudio Pinhanez. 2005. A Study on the Manipulation of 2D Objects in a Projector/Camera-Based Augmented Reality Environment. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (Portland, Oregon, USA) (CHI '05)*. Association for Computing Machinery, New York, NY, USA, 611–620. <https://doi.org/10.1145/1054972.1055056>
- [27] Robert Xiao, Miguel A. Nacenta, Regan L. Mandryk, Andy Cockburn, and Carl Gutwin. 2011. Ubiquitous Cursor: A Comparison of Direct and Indirect Pointing Feedback in Multi-Display Environments. In *Proceedings of Graphics Interface 2011 (St. John's, Newfoundland, Canada) (GI '11)*. Canadian Human-Computer Communications Society, Waterloo, CAN, 135–142.